



École Polytechnique

BACHELOR THESIS IN COMPUTER SCIENCE

Computer Graphics: First project

Anatole Jacquin de Margerie, École Polytechnique

Academic year 2024/2025

I want to preface this by saying that I had lots of fun, and even if I did get stuck, I enjoyed problem solving and seeing the fruit of my labor! I also noted throughout that I always needed to have a higher light intensity to obtain similar results to those shown in the slides, but I do not think this is a big issue.

For TD1, I succesfully implemented transparency and mirrors, but after struggling for several hours on the hollow ball, I had to give up, either obtaining a fully black ball, or a weird mirror in the center of the ball and dark on the contours...

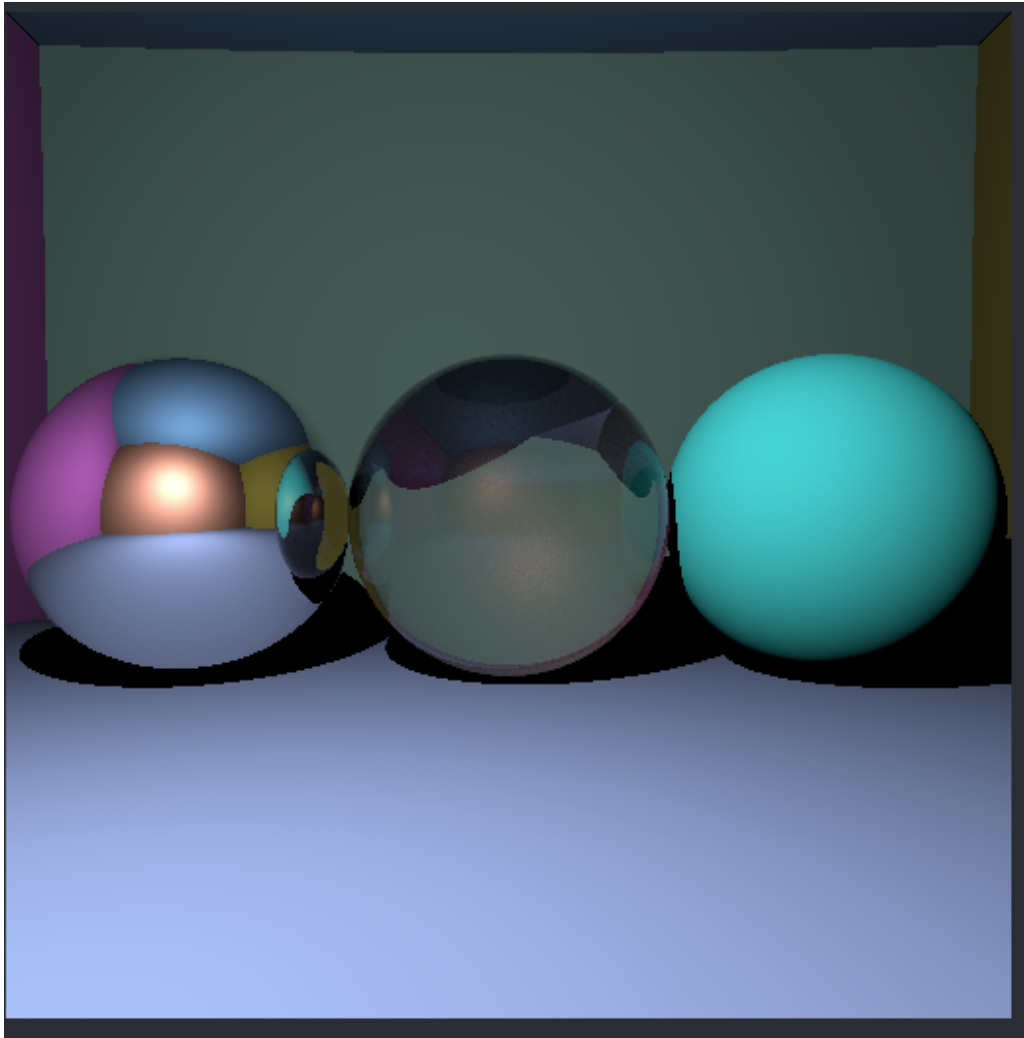


Figure 1: Mirror, transparent and Diffuse balls rendered with 1000 rays per pixel, 512x512 image, rendered in 40 secs

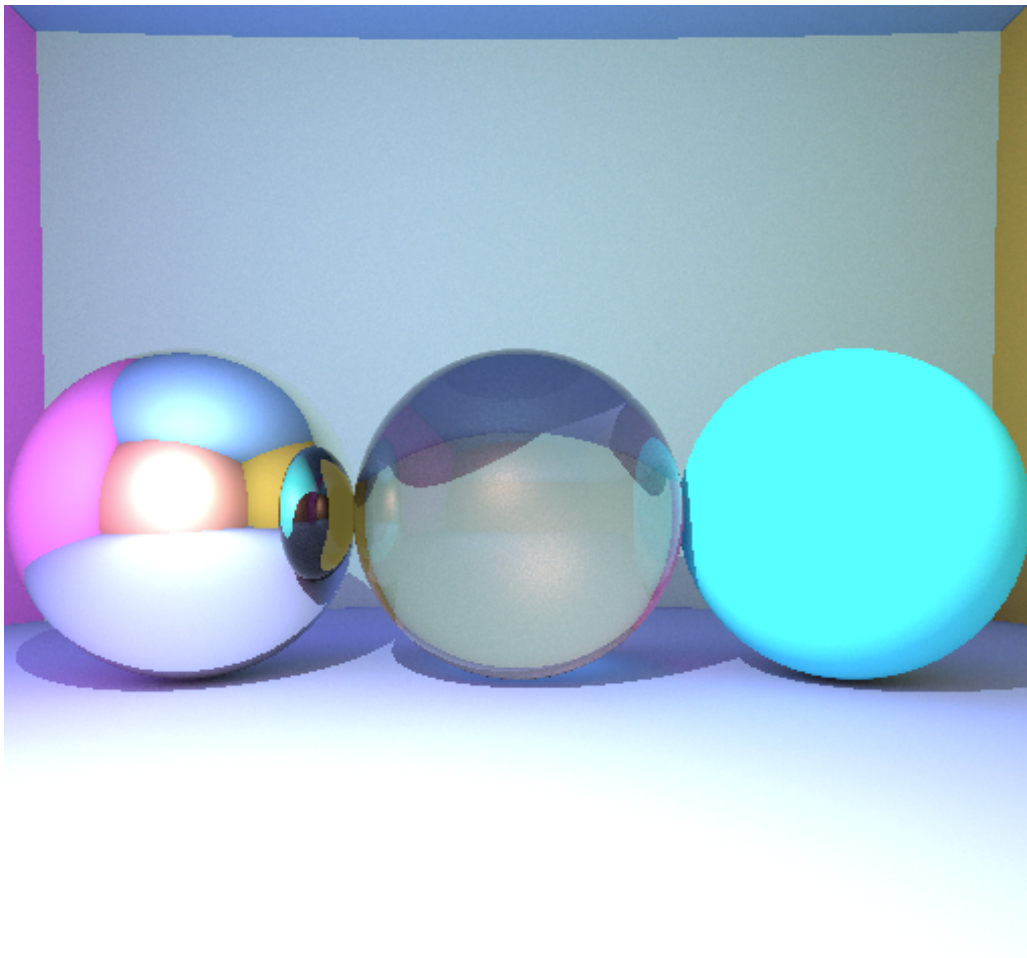


Figure 2: Mirror, transparent and Diffuse balls rendered with 1000 rays per pixel, 512x512 image, with indirect lighting, rendered in 3m47 secs, (28secs with parallel)

Indirect lighting didn't take too long for me to implement, however it implied revamping the majority of my code, as I had not built a scene class (instead, it was just a Vector list), and so I had to re-adapt most functions to become functions of the scene class, which probably made things easier down the road. I only implemented the Lambertian diffuse version however (even though glossy surfaces looked fun to try). We note that indirect lighting adds a lot of rendering time.

Anti-aliasing, using Box-Muller, was also quick to implement, and the results did not slow down rendering much, despite showing good results.

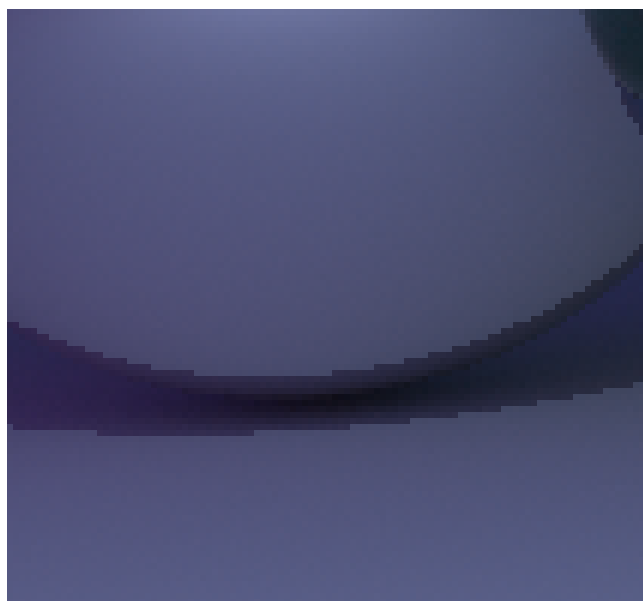


Figure 3: before anti-aliasing



Figure 4: after anti-aliasing

Finally the rendering of triangle-mesh was quite difficult for me. I firstly really struggld with the Geometry class, and adapting the sphere and triangle mesh classes to it. I managed eventually to get a cat to render, although it initially was missing triangles for some reason, leading to holes in the shape, or on the contrary, I also had it completely covering the image...



Figure 5: Triangle mesh rendering of cat, 32 spp, rendered in 18m34 with parallelism. Reduced to 2m39 with ray-bounding box testing, 3.2secs with BVH

I managed to get the bounding box logic to work and function correctly, and got good speedup, however the BVH really stumped me. For some reason the recursive logic didn't terminate, no matter what I tried, and I just couldn't seem to debug it. Because no friends seemed to see the problem with my implementation, I did ask chatgpt for help to find the issue, and it finally managed to terminate. Therefore, although I understand the BVH code and spent time on it, the final version was not written by me, and so I would like to not be graded on it...

Many thanks for your time, and for this fun course!